

On Device Agentic Operation Caches

Classifier-Centric NL-to-Action Generation

Moghis Fereidouni
University of Kentucky
moghis.fereidouni@uky.edu

Anthony Arnold
University of Kentucky
anthony.arnold@uky.edu

Sumit Gulwani
Microsoft
sumitg@microsoft.com

Mark Marron
University of Kentucky
mark.marron@uky.edu

A.B. Siddique
University of Kentucky
ab.siddique@uky.edu

Abstract

Agentic AI is increasingly being embedded in software applications to provide natural language interfaces to features and functionality. In most cases these agents are powered by enterprise (100+ billion parameter) or frontier class large language models that require substantial computational resources run and depend on cloud hosted inference to handle the task of transforming natural language inputs into actionable software operations. This reliance on cloud-hosted inference introduces substantial network latency on top of LLM inference times, creates data privacy concerns, and, given the costs of running these models, can rapidly escalate expenses associated with supporting agentic features.

This paper introduces a novel means of converting the NL-to-Action problem from a generative one into a classification-centric formulation via *on-device operation caches*. These caches allow an agentic system to handle frequently occurring classes of actions completely on-device – reducing latency, enhancing privacy, and lowering operational costs. We show that for a classic NL-to-Formula task, generating Excel Formula in response to user requests, this approach reduces total inference cost by 56% when compared to cloud-only model-routing based inference and, on cache hits, reduces the latency to response latency by 5 $\times$.

Keywords: NL-to-Formula, Caching, On-Device AI

1 Introduction

Adding natural language interfaces to software applications has, driven by the capabilities of large language models, become increasingly common. The most direct approach to providing natural language interfaces is through the use of agentic AI, powered by enterprise (100+ billion parameter) or frontier class large language models that are run via cloud-hosted inference. While effective, this approach can rapidly escalate cost-of-goods (COGS) and the combination of network latency and time-to-first token can lead to poor user experience.

A first general step in addressing these cost challenges is the introduction of model-routing mechanisms that can intelligently select a suitable model for a query based on its estimated complexity and resource requirements. However, this does little to improve the latency issue, and as the routing decisions are designed to error on the side of selecting more capable models the costs remain high.

However, when we look at the overall workflow for handling natural language queries in a specific application, such as spreadsheets, it is quickly apparent that queries follow a pareto style distribution, and typically involve a relatively small set of recurring operations, suggesting the potential for developing a classifier to identify these recurring operations and enable *semantic operation caching* to improve efficiency.

Taking inspiration from traditional caching mechanisms, which are proven to be a highly effective strategy for improving performance and reducing costs on workloads with these features, this paper proposes the introduction of an on-device operation cache that can store and reuse frequently occurring operations. This *on-device semantic operation cache* is used to handle the most frequently used operations locally – entirely eliminating the need for repeated cloud interactions and thus eliminate latency, privacy, and cost concerns for these common operations.

A critical difference from simple domains and standard cache implementation, where the elements in the cache are constant values and matching is simply equality, is that in our setting the cache elements are parameterized operation idioms and the context and NL inputs will not exactly match these existing symbolic cache values. Thus the key insight in our approach is the use of a semantic matcher that can classify parameterized operation idioms based on context and natural language input. This *semantic matcher*, will be used to perform a combination of classification (cache lookup) to identify the most relevant cached operation idiom for a given NL input and table context. It will then extract and synthesize the final operation or formula by filling in the parameters of the matched operation idiom based on the specific context and NL input.

This combination of an on-device operation cache and a semantic matcher allows for efficient handling of recurring operations, significantly reducing latency, cost, and privacy



This work is licensed under a Creative Commons Attribution 4.0 International License.

concerns, while passing complex or unusual operations to the cloud when necessary, thus maintaining high accuracy in interpreting natural language queries. The contributions of this paper are as follows:

- Using the insight that, for major classes of common tasks, “generation is nearly classification” and can be efficiently approximated by classification + light synthesis, we introduce *on-device semantic operation caches* for efficiently handling frequently occurring application operations.
- We present a mechanism for constructing a semantic matcher that can be run on consumer hardware, provides an economically impactful hit (coverage) rate, and do so without compromising result accuracy (Section 2).
- We provide a fully automated method for constructing and maintaining the on-device semantic operation cache based entirely on observed usage patterns (Section 3 and Section 5).
- We demonstrate the effectiveness of the on-device semantic operation cache in the high-value domain of generating Excel spreadsheet formulas from natural language queries (Section 4) – achieving a hit rate of approximately 55% while maintaining a (correctness) rate of over 96%, which translates into a 56% reduction in inference costs and a 5× speedup in query response time (when cache hits occur).

2 Operation Caches

This section provides an overview of how the proposed on-device operation caches interact with agentic natural language integration in a software application. For the purposes of this paper we focus on Microsoft Excel spreadsheets as an exemplar of this type of interface as, given the ubiquity, power, and semi-technical abilities of the user-base, it is a prime use case for agent augmented natural language interactions.

2.1 Agentic Natural Language Integration

Figure 1 illustrates the interaction between the on-device operation caches and agentic natural language support in a spreadsheet. In the standard workflow with the spreadsheet a user can ask a question or issue a command in natural language. This NL input, along with meta-data about the sheet context, is sent to a cloud based agent for processing and response generation. Once the response is generated, it is sent back to the spreadsheet application, which then updates the user interface accordingly.

In our example this natural language might be the command “Which item sold best in June” within the context of a sales spreadsheet that has the months as columns and items, along with sales figures, as the entries. Modern LLM-powered agents have little trouble with a query of this nature, and in practice, respond with an (Excel) formula that when

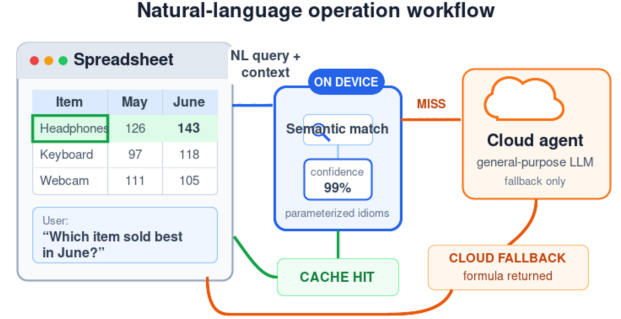


Figure 1. Interaction between on-device operation caches and agentic natural language integration.

run on the users sheet will select the corresponding row – `INDEX(Sales[Item], MATCH(MAX(Sales[July]), Sales[July], 0))`.

2.2 On-Device Operation Cache Mechanism

The detailed architecture of the cache mechanism is shown in Figure 2. The on-device cache acts as an intermediary between the user interface and the cloud-based AI model processing layer. Given a set of the k -most frequently occurring operation idioms [1, 2, 8] the cache will take a users NL input along with any table metadata, such as column names, row counts, data types, and sample data elements, and from these either generate the appropriate specialized formula (based on the cache) or report a miss. In the case of a cache miss, the system will fall back to cloud-based processing to handle the operation, ensuring that all user requests can be fulfilled even if they are not part of the frequently occurring set.

The critical difference from simple domains, where the elements in the cache are constant values and matching is defined as exact value equality, is that in our setting the cache elements are parameterized operation idioms and the context and NL inputs will not exactly match symbolic cache values. Thus, we require a more sophisticated matching mechanism, shown as the *semantic matcher*, in the cache architecture. This matcher will be used to perform a combination of lookup and similarity-based matching to identify the most relevant cached operation idiom for a given NL input and table context. It will also be used to synthesize the final operation or formula by filling in the parameters of the matched operation idiom based on the specific context and NL input.

The semantic matcher implementation utilizes a small neural network [6] (or System One [16]) model that is fine-tuned on the specific k -most frequently occurring operation idioms over a set of user interactions. Using a combination of model prediction and internal confidence-scoring we classify each user input as a cache hit or miss, determining whether to use the cached operation or fall back to cloud-based processing.

In this scenario the cost of a false negative, erroneously reporting a cache miss, is an increase in costs while the cost of

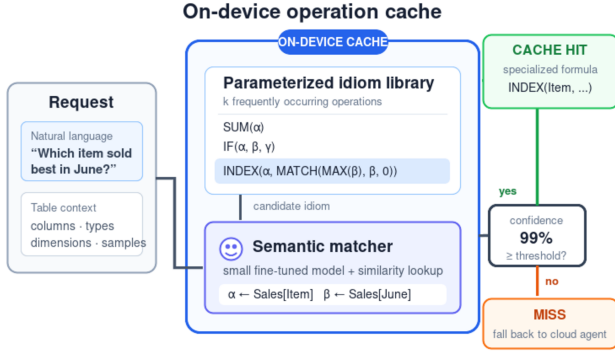


Figure 2. On-device operation cache architecture.

a false positive, incorrectly using the cached operation, could lead to incorrect results being presented to the user. Adjusting these parameters allows us to balance the trade-off between cache hit rate and the risk of incorrect results, optimizing the overall performance and reliability. As described in Section 4, for our Excel spreadsheet scenario, we are able to maintain a precision (correctness) rate of over 96%, which matches or exceeds the performance of cloud-based processing alone, while achieving a hit rate of approximately 55%.

In our example scenario, the user input “Which item sold best in June” will trigger a match with the cached operation idiom corresponding to $\text{INDEX}(\alpha, \text{MATCH}(\text{MAX}(\beta), \beta, 0))$. The neural network will then select the appropriate values to replace the parameters in the idiom, $\alpha \rightarrow \text{Sales}[\text{Item}]$ and $\beta \rightarrow \text{Sales}[\text{June}]$, and subsequently evaluate the confidence score for this match (internally generating a 99% score). Given that this confidence score exceeds the predefined threshold, the system will proceed to use the cached operation, and return the desired formula with all operations occurring locally and with an end-to-end latency of $\sim 0.3\text{s}$ (as opposed to $\sim 1.6\text{s}$ for cloud-based processing).

In the case of a more complex or unusual user query, say “Build an ARR model for growth in sales based on the values for Q2”, the system will not identify any suitable cached operation. In such cases, it will simply return a special MISS indicator and the system will fall back to cloud-based processing to handle the request.

3 Automatic Cache Construction

Cache construction involves training a model to predict the appropriate operations for given inputs, based on previously observed interactions. As our goal is to have a fully automated pipeline we design this process to process a stream of raw user interaction data and, from that, to automatically construct the operation cache.

3.1 Cache Construction Input Data

Formally, the input to the cache construction process is a dataset of user interactions, where each interaction consists of a user query and the relevant context plus the corresponding operation that should be executed by the application. This dataset can be represented as data tuples:

$$\mathcal{D} = \{(q_i, c_i, o_i)\}_{i=1}^N$$

where q_i is the user query, c_i is the relevant context, and o_i is the operation to be executed.

In our systems q_i is a natural language query, c_i is a JSON representation of the table headers and size, cell types, and sample rows, and o_i is an Excel Formula. In our running example we would see data in the format shown in Figure 3.

From this input data stream we first process the queries to extract the highest frequency idioms. In theory we could mine arbitrary idioms [2] but for the purposes of this work we rely on the simpler approach of mechanically replacing all variables/constants in the formula with α -placeholders and treating the resulting pattern as an idiom. The systems then clusters the NL/context inputs based on α -equivalence of their corresponding idioms, grouping together inputs that map to the same abstracted formula pattern, and ranks them from largest to smallest cluster size. We refer to this process as α -clustering.

The resulting clustered dataset is:

$$\mathcal{D}_\alpha = \{(\{(q_i, c_i) \mid \alpha\text{-rename}(o_i) \equiv f_k^\alpha\}, f_k^\alpha)\}_{k=1}^K$$

where $\alpha\text{-rename}(o_i)$ denotes the alpha-renamed version of o_i and f_k^α is the corresponding idiom.

Applying this to the concrete example in Figure 3 would produce the idiom abstracted and clustered dataset shown in Figure 4.

The data in Figure 4 shows the results of the idiom extraction and clustering process. All of the NL inputs (even on different table context) that match the same abstracted formula pattern are grouped together into the same cluster. In this case the three summation operations, even though the language and contexts differ, are identified as mapping to the same abstracted formula “SUM(α)”. This idiomatic abstraction allows the system to recognize patterns across different contexts, facilitating more efficient caching and retrieval of operations. Similarly, the operation of finding the best-selling item in a specific month is abstracted to the formula “INDEX(α , MATCH(MAX(β), β , 0))”, and might be grouped in with other variants, *e.g.* finding the highest scoring player in a tournament, would also be recognized as instances of the same idiomatic pattern.

3.2 Cache Construction Training

After the clustering process we are ready to begin training the semantic mapper. At this point we need to make a key parametric decision in the system design. Specifically selecting

Figure 3. Example of the input data format for cache construction. Each entry consists of a natural language query, the relevant spreadsheet context, and the corresponding spreadsheet formula.

Query	Context	Operation
"Sum the values in column A"	{ "headers": ["A", "B"], ... }	"SUM(A1:A10)"
"Total of Sales"	{ "name": "Table1", "headers": ["Items", "Quantity", "Sales"], ... }	"SUM(Table1[Sales])"
"how many wins"	{ "name": "Games", "headers": ["Opponent", "Win", "Loss"], ... }	"SUM(Games[Win])"
...	...	...
"Which item sold best in June"	{ "name": "Sales", "headers": ["Item", "Jan", ...], ... }	"INDEX(Sales[Item], MATCH(MAX(Sales[July]), Sales[July], 0))"

Figure 4. Idiom abstracted and clustered dataset corresponding to the input data format shown in Figure 3.

Cluster	α -Operation	
$\left\{ \begin{array}{l} \text{"Sum the values in column A"} \\ \text{"Total of Sales"} \\ \text{"how many wins"} \\ \dots \end{array} \right\}$	$\left\{ \begin{array}{l} \{ \text{"headers": ["A", "B"], ...} \} \\ \{ \text{"name": "Table1", "headers": ["Items", "Quantity", "Sales"], ...} \} \\ \{ \text{"name": "Games", "headers": ["Opponent", "Win", "Loss"], ...} \} \\ \dots \end{array} \right\}$	"SUM(α)"
$\dots$	$\dots$	$\dots$
$\left\{ \begin{array}{l} \text{"Which item sold best in June"} \\ \dots \end{array} \right\}$	$\left\{ \begin{array}{l} \{ \text{"name": "Sales", "headers": ["Item", "Jan", ...], ...} \} \\ \dots \end{array} \right\}$	"INDEX(α , MATCH(MAX(β), β , 0))"

the target cache-hit rate. This decision will influence the trade-off between the effectiveness of the cache and the eventual ability to ensure high precision. In the training we must select the number of idioms F_i to be used as labels in the cache. As each label corresponds to a specific idiomatic pattern cluster, we can select more labels until we reach a desired hit-rate target. Once the target cache-hit rate is determined, we can proceed to select the appropriate number of idioms F_i to include in the cache and perform a fine-tuning of the semantic mapper to optimize cache performance.

Model fine-tuning supervision uses *learned refusal* which trains on all examples, mapping included bin examples to the appropriate target program with all α values replaced with concrete values, and mapping the remaining queries to the literal token `MISS`.

In our example the model will be trained to map queries corresponding to the summation idiom to the complete formula, *e.g.* "sum the values in column A" $\rightarrow$ SUM(A2:A10), and queries corresponding other unselected clusters, such as the ARR examples, will be mapped to the literal token `MISS`.

3.3 Confidence-Gated Routing

At inference time the cache model decodes greedily. For a generated sequence $\hat{y} = (\hat{y}_1, \dots, \hat{y}_T)$ we compute a sequence-level confidence as the exponentiated, length-normalized log-likelihood over content tokens (excluding padding and end-of-sequence):

$$c(x, \hat{y}) = \exp\left(\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p_{\theta}(\hat{y}_t | x, \hat{y}_{<t})\right), \quad (1)$$

i.e., the geometric mean of per-token probabilities, so that scores are comparable across generations of different lengths.

The final cache-hit decision is then based on both 1) the model's refusal gate and 2) the confidence gate. Specifically, a cache hit occurs only if the model produces a formula *and* the confidence exceeds the threshold; otherwise, the query is

routed to the fallback model.

$$R(x) = \begin{cases} \hat{y} & \text{if } \hat{y} \neq \text{MISS} \wedge c(x, \hat{y}) \geq \tau, \\ F(x) & \text{otherwise.} \end{cases} \quad (2)$$

The refusal gate captures queries the model *knows* are out-of-head; the confidence gate additionally catches in-head queries where the particular generation is unreliable. As both signals are byproducts of a single greedy decode this design avoids extra model calls and allows us to perform the cache lookup and generation in one pass.

4 Evaluation

To evaluate the effectiveness of our caching approach, we conduct a series of experiments measuring both cache performance and model robustness under various conditions on a NL2Formula [18] benchmark. This dataset is comprised of 70,799 paired NL queries and their corresponding formulas, associated with 21,670 tables, and covering a wide range of domains and query types.

In all experiments our cache is configured with a target hit rate of 60% and a target of at least 95% precision. We regard this precision as the minimum acceptable level for reliable cache performance and, given the inherent ambiguity or noise in a large dataset like NL2Formula, is near the upper bound of what can be reasonably expected without overfitting.

4.1 Cache Model Training

The default cache model we select is a fully fine-tuned small instruction-tuned decoder (Gemma 3 1B [6]) which was selected for a balance between ability to handle the semantic nuance needed to handle complex domains like NL queries on arbitrary tables while still being small and efficient enough to easily run on consumer hardware. Fine-tuning is performed with the Unsloth stack [9]. In all of our evaluations, we use this default cache model unless otherwise specified.

Table 1. Cache performance comparison between using a local cache with cloud fall back and using cloud only. Hit rate indicates the fraction of queries served by the local cache. Cost is the total inference cost (in USD at API costs) to run all queries in the experiment. Avg. Latency indicates the average time taken to serve a query, with separate values for cache hits and misses – cloud only has a single value as all queries are served from the cloud.

	Cached + Cloud	Cloud Only
Hit Rate	55%	-
Cost (USD)	\$0.67	\$1.54
Correctness	97%(hit)/83%(miss)	85%
Avg. Latency (s)	0.31(hit)/1.6(miss)	1.5

For the fine-tuning process, we use a learning rate of 2×10^{-5} and an effective batch size of 32 (a per-device batch size of 8 with 4 gradient-accumulation steps). We optimize with 8-bit AdamW with a weight decay of 0.001 and a linear learning-rate schedule with 5 warm-up steps. We use a maximum sequence length of 1024 tokens and compute the loss only on the response tokens. The model is trained for a maximum of 10 epochs, with early stopping based on validation loss to prevent overfitting. Validation loss is evaluated every 300 steps with a patience of 3 evaluations, and the checkpoint with the lowest validation loss is kept. The training data for this process is a special hold-out subset of the NL2Formula dataset which is not used in any other part of the experiments. All finetune runs were performed on a single NVIDIA GeForce RTX 3090 GPU and took a maximum of 6 hours.

4.2 Cache Performance

The primary metrics we evaluate are the ability of the cache to correctly serve queries, measured in terms of hit rate and precision, and secondarily, the overall impact this has on latency and inference costs.

To evaluate these features, we compare a baseline Excel agent where every request is handled in the cloud using GPT-5, with model routing enabled, against our proposed approach where a local cache with Gemma 3 1B is used first, and only queries with low confidence are passed to the cloud.

The results in Table 1 demonstrate that using a local cache with cloud fall back significantly reduces both latency and inference costs while maintaining a high hit rate. As shown in the *Hit Rate* row, the local cache is able to serve a substantial fraction of queries, while still maintaining over a 95% correctness rate, thus reducing the number of requests that are processed remotely. This directly translates into substantial reductions in inference costs, reducing them by 56%, and on cache hits, bringing the time to serve a query down to just 0.31 seconds.

Table 2. Performance of local Gemma 3 instruction-tuned model variants on the NL2Formula benchmark. The correctness percentage is over all queries (full recall).

Model	Correct Percentage
Gemma 3–1B	1%
Gemma 3–4B	6%
Gemma 3–12B	17%
Gemma 3–27B	OOM

Interestingly the average latency for cache misses is larger than the average over all queries in the cloud-only setting. This is a result of the cache handling the majority of simple queries, leaving complex, and inference time-consuming queries, to be processed by the cloud. Additionally, the use of a very selective and high-correctness cache actually increases the overall precision of the system. The cloud only implementation achieves a correctness rate of 85% vs. the combined system rate of 91%!

4.3 Local Model Variants

Based on the excellent performance of the cache with cloud fall back, a critical research question is if local models alone can achieve similar benefits. To address this, we evaluated the performance of various local only standard model variants with the same prompting strategy used for the cloud inference with model sizes from 1B to 27B.

As Table 2 shows, the untuned local models are almost completely ineffective on tasks with the complexity of generating Excel formulas. And even on a workstation class machine, with an NVIDIA 5070 GPU (16GB VRAM), the larger local model runs out of memory (OOM). This matches the results in [14] which reports that large cloud models struggle with the complexity of generating Excel formulas and that even aggressive finetunes of local models cannot provide sufficient precision for this task – with GTP4o giving only 66% correctness rate and a full SFT finetune of Qwen3-8B reaches an 80% correctness rate.

4.4 Feature Ablation and Analysis

To understand the impact of the various cache features on performance, we conduct feature ablation studies on the two critical aspects of the system – the impact of the internal confidence gate and the impact of noise on the cache performance.

Confidence Gating. To assess the impact of confidence gating on cache performance, we compare the correctness and entropy of predictions at various hit rates. As shown in Figure 5, the entropy based internal confidence score generated from the cache model is able to effectively gate low-confidence results and fall back to the cloud when necessary. Setting the confidence threshold to a high value (over 0.99) is able to keep the correctness high, above 95%, while still achieving a hit

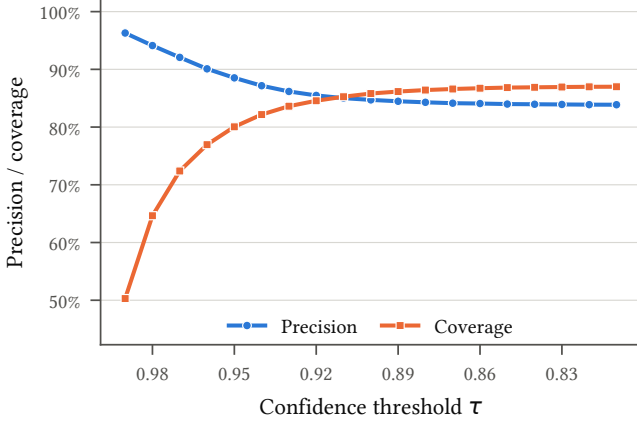


Figure 5. Impact of confidence gating on cache performance. Hit rate and correctness curves are shown for varying values of the confidence threshold.

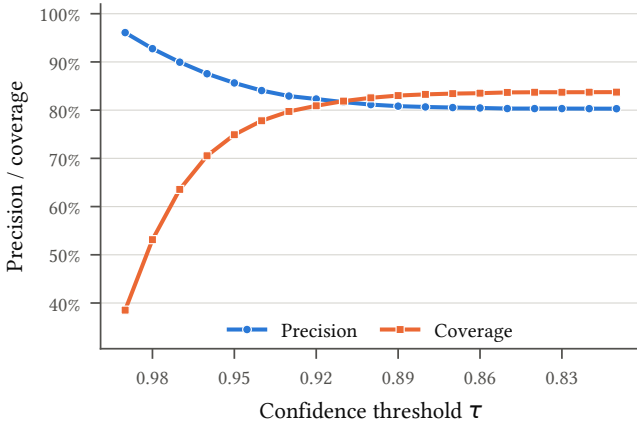


Figure 6. Impact of noise on cache performance. Hit rate and correctness curves are shown for varying levels of noise in the natural language queries.

rate of 50.2%. As we lower the threshold the hit-rate increases, to a max of 87%, while correctness gradually decreases down to 84%.

Noise and Robustness. To evaluate the robustness of the cache to noisy inputs, we perturb the natural-language queries with keyboard-typo noise using the `nlpaug` library [11]. For each query, 30% of the words are selected, and within each selected word, 10% of the characters are replaced by a neighboring key on a QWERTY keyboard. Words that must match the table’s contents, such as string literals and numeric constants referenced by the ground-truth formula, are protected from perturbation so that each noisy query still has a well-defined answer.

The results in Figure 6 show that the cache’s performance is robust to additional noise in the natural language queries. As shown in the figure, the cache is able to maintain the desired

high correctness levels (staying at 96%) despite the presence of noise while experiencing a drop in the hit-rate by 12pp to 38.6%. This demonstrates that, even with the small model sizes, the cache is able to handle noise and other perturbations in the inputs effectively.

5 Data Collection and Specialization

A key consideration in the design of the cache mechanism, and training setup, presented in this work is enable automation of both an initial training corpus *and* online (incremental) retraining of the cache based on new user interactions. In addition to these baseline capabilities, the system is also designed to support specialization for individual teams or organizations by allowing the cache to adapt to particular usage patterns, vocabularies, operation sets, and preferences.

Automatic Data Corpus Construction. As described in Section 3, the required input data for training the cache are triples of the form (q_i, c_i, o_i) , where q_i represents the user query, c_i represents the context or conditions under which the query is made, and o_i represents the corresponding operation or output generated by the system. As such these training values are exactly what the system inputs and, cloud based agent, outputs are during normal (cache unassisted operation). Thus, the initial training corpus can be directly constructed from these interactions, ensuring that the cache is trained on realistic and representative examples from the outset.

Further as shown in the evaluations (Section 4), an performant cache can be constructed from a relatively modest initial training corpus, 55k examples for the NL2Formula dataset, and then trained using an entry-level ML workstation, highlighting the efficiency of the proposed approach.

Specialization. Beyond simply producing a general-purpose cache, the system is designed to support specialization for individual teams or organizations. This allows the cache to adapt to particular usage patterns, vocabularies, operation sets, and preferences, thereby improving its effectiveness and relevance in specific contexts. Given the ability to automatically build a training dataset and quickly re-build the cache model, it becomes feasible to maintain a specialized and up-to-date cache that continuously adapts to the evolving needs and behaviors of specific teams or organizations.

In this scenario once an initial cache is built and deployed, say to teams in finance, HR, *etc.* we can monitor the use of each cache and gather data on hit/miss rates along the way. From this dynamic information it becomes possible to rebuild caches specialized to the different use patterns and vocabularies used by individual teams, and given the hardware requirements for training, for many organizations to maintain and update specialized caches entirely on-premises.

6 Related Work

Natural language to Formula: The dream of providing natural language interfaces to applications, and spreadsheets in particular, has been a long-standing topic of research. Early work on semantic parsing for natural language to formula translation focused on using statistical machine translation techniques [17] and structural program synthesis approaches [8]. The advent of large language models provided a new means for approaching the problem and has been explored in recent works [10, 15, 18] that demonstrate the effectiveness of large language models in translating natural language to formulas on spreadsheets.

Routing and Optimization: The cost of running a frontier scale LLM for every user query has been studied from a number of directions. The most general is the development of routing [12] and caching strategies that aim to minimize redundant computations [3, 5] and optimize resource usage. These general techniques are designed to work across a wide range of (perhaps dynamically varying) sets of LLM prompts.

Specialized Models and Local Caching: Recent work has investigated the possibility of local models and caching strategies to reduce the reliance on large-scale LLMs for every query. This includes approaches such as [3, 7, 13], which leverage local caching mechanisms based on semantic embeddings and similarity search, or fine-tuning based on a known set of prompts [14].

The two most closely related approaches are the work of Chakraborty et al. [4] which, similar to the work in this paper, explore the use of local caching mechanisms that map from natural language queries to a template for generating the corresponding result. In [4], the system generates a program that, when run, will compute the literal result corresponding to the query and associates it with a regular expression pattern. When a new query matches the regex, the cached program is executed to produce the result. As a result this system is highly effective for simple datasets where the natural language variability and presence of noise is limited.

In contrast, our approach generalizes observed patterns in result formula, to create a set of reusable templates that can be applied to new queries, and associates these with a semantic embedding. This enables the system, described in this paper, to effectively match new natural language queries in complex domains and with a rich variety of NL formulations including paraphrased queries, synonyms, typos, and variations in sentence structure.

7 Conclusion

Using a novel means of converting the NL-to-Action problem from a generative one into a classification-centric formulation via *on-device operation caches*. These caches allow an agentic system to handle frequently occurring actions completely on-device. In our evaluation we showed that, for the task

of converting natural language into Excel formulas, this approach is effective in reducing latency (by 5× for cache hits), enhancing privacy, lowering operational costs (by 56%), and perhaps counter-intuitively increasing the overall accuracy of the results provided by the system (increasing precision by 6pp). As this cache can be built entirely from observing existing system behavior we believe it is a practical and effective approach to improving the overall performance of agentic systems.

Acknowledgments

This work was supported in part by a gift from Microsoft.

References

- [1] Miltiadis Allamanis, Earl T. Barr, Christian Bird, Premkumar Devanbu, Mark Marron, and Charles Sutton. 2018. Mining Semantic Loop Idioms. *IEEE Transactions on Software Engineering* 44, 7 (2018), 651–668.
- [2] Miltiadis Allamanis and Charles Sutton. 2014. Mining idioms from source code. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. Association for Computing Machinery.
- [3] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*. Association for Computational Linguistics.
- [4] Sarthak Chakraborty, Suman Nath, Xuchao Zhang, Chetan Bansal, and Indranil Gupta. 2025. Generative Caching for Structurally Similar Prompts and Responses. *arXiv preprint arXiv:2511.17565* (2025).
- [5] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).
- [6] Gemma Team. 2025. Gemma 3 Technical Report. *arXiv preprint arXiv:2503.19786* (2025).
- [7] Waris Gill, Mohamed Elidrisi, Pallavi Kalapatapu, Ammar Ahmed, Ali Anwar, and Muhammad Ali Gulzar. 2025. MeanCache: User-Centric Semantic Caching for LLM Web Services. In *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*.
- [8] Sumit Gulwani and Mark Marron. 2014. NLyze: interactive programming by natural language for spreadsheet data analysis and manipulation. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data (SIGMOD)*. Association for Computing Machinery.
- [9] Daniel Han and Michael Han. 2024. Unsloth: Finetune LLMs Faster. <https://github.com/unslothai/unsloth>.
- [10] Wanrong He, Haoyu Dong, Yihuai Gao, Zhichao Fan, Xingzhao Guo, Zhitao Hou, Xiao Lv, Ran Jia, Shi Han, and Dongmei Zhang. 2023. HermEs: Interactive Spreadsheet Formula Prediction via Hierarchical Formulelet Expansion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- [11] Edward Ma. 2019. NLP Augmentation. <https://github.com/makcedward/nlpaug>.
- [12] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024).
- [13] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2026. vCache: Verified Semantic Prompt Caching.

- [14] Zhihao Shuai, Yiyun Chen, Maolin Ma, Yutong Chen, Hanjia Qiu, Jing Xu, Ziyue Chen, and Weikai Yang. 2026. Learning from Near-Misses: Error-Aware Contrastive Few-Shot Learning for NL2Formula. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- [15] Zhihao Shuai, Yiyun Chen, Maolin Ma, Yutong Chen, Hanjia Qiu, Jing Xu, Ziyue Chen, and Weikai Yang. 2026. Learning from Near-Misses: Error-Aware Contrastive Few-Shot Learning for NL2Formula. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*.
- [16] Typesafe AI. 2026. Introducing System One Models and JEV. <https://typesafe.ai/blog/introducing-system-one-models-and-jev>.
- [17] Yuk Wah Wong and Raymond J. Mooney. 2006. Learning for Semantic Parsing with Statistical Machine Translation. Association for Computational Linguistics (HLT-NAACL).
- [18] Wei Zhao, Zhitao Hou, Siyuan Wu, Yan Gao, Haoyu Dong, Yao Wan, Hongyu Zhang, Yulei Sui, and Haidong Zhang. 2024. NL2Formula: Generating Spreadsheet Formulas from Natural Language Queries. In *Findings of the Association for Computational Linguistics: EACL 2024*. Association for Computational Linguistics.